AgentWorkShop: A Modular Task-to-Device Integration Framework for Industrial Agents

***Abstract*—Industrial agents need a software path that connects a task to production context, semantic operations, and configured device interfaces. AgentWorkShop is a modular integration framework that connects collaborative tasks through a registered harness interface to production context, per-call operation checks, semantic I/O, configured protocol drivers, and service-owned records. Its shared dispatch contract separates tool discovery from authority checks; route handlers bind approval to the active line–product–recipe–run state, and the system records command receipts separately from later process observations. To assess route execution and record continuity, we evaluated a registered-harness task route and a direct operation route in cast-film and annealing simulators with different signal and protocol mappings. Six sequential tasks passed task-to-approval-to-receipt-to-sample linkage checks, although none of their snapshots reached its target band. A deterministic fixture replay passed 31 assertions. Two direct calls returned verified receipts and run-tagged snapshots, also outside target. A mock-state suite refused 12 stale-context changes across three recipe routes; an ablation without tuple equality allowed six product or run changes to trigger mock-driver writes. The results verify selected integration paths and the added refusal coverage of one check; they do not establish process attainment, atomicity during device I/O, cross-harness portability, or comparative platform benefit.**

***Index Terms*—Industrial agents, industrial communication, industrial informatics, production systems, semantic data integration.**

I. Introduction

INDUSTRIAL-AGENT deployments must connect each task to its production context, process signals, and equipment interfaces. Existing work addresses end-to-end automation control, operator-reviewed control-sequence generation, modular factory skills, and production scheduling [**?**], [**?**], [**?**], [**?**]. Other studies target PLC code generation, context-aware control, resource capability discovery, shop-floor coordination, digital-twin fault handling, natural-language machine access, and robotic-line control [**?**], [**?**], [**?**], [**?**], [**?**], [**?**]. Across production lines, agent roles, signal maps, protocols, objectives, and product/run context vary. An integration architecture must accommodate these differences around shared task and operation services while keeping each action tied to its active production context. Recent industrial-agent architectures add AAS-grounded OPC UA actions with signed approval, P&ID-grounded proposal checks, and governed sensor–MES–ERP evidence streams [**?**], [**?**], [**?**]. These works address asset-specific execution, process validation, or closed-loop analytics. This paper addresses the complementary software-integration question of composing task and harness routes, production context, semantic protocol I/O, and route-owned records.

Consider two production scenes with different process signals and interfaces: cast-film thickness control and annealing-zone temperature control. An agent task in either scene needs production context, typed signal access, and a route from a named operation to the configured device protocol. The framework must also distinguish a command receipt from a subsequent process observation. The shared task and operation services can therefore be configured with scene-specific signal maps, protocols, and limits while retaining a common execution path. The evaluation instantiates this path in two simulator scenes.

This paper presents AgentWorkShop as a modular framework for task-to-device integration in industrial agents. It connects collaborative tasks through a registered harness interface to production context, manufacturing execution system (MES) data, analysis and modeling services, data-acquisition (DAQ) and write-control (DCW) nodes, and protocol-specific drivers. Its interfaces include command-line, web, software development kit (SDK), Representational State Transfer (REST), Model Context Protocol (MCP), and Agent-to-Agent (A2A) surfaces. Task and chat routes enter the collaboration runtime, whereas a direct REST tool-invocation route can call the shared operation dispatcher without creating a task. Scene blueprints define signal maps, objectives, and guards; channel profiles, tool policies, and agent bindings scope operations. Agents and callers invoke named operations through shared runtime checks; the framework does not choose routes autonomously and retains proposals, command confirmation, and process observations as distinct records. Fig. 1 presents the system architecture, and Fig. 2 shows the cross-module task flow. Section IV details the node-exploration and recipe lifecycles.

This paper asks whether task and direct-operation routes can execute bounded operations in two simulator scenes with different signal and protocol mappings while retaining route-specific approval, receipt, and observation records. It also tests whether approved recipe actions are refused when their active context changes before software dispatch. A deterministic fixture separately replays the registered-harness route without external model decisions. A mock-state suite injects line, product, recipe, and run changes during approval for `recipe_apply`, `recipe_trial`, and dispatching `recipe_rollback`; all stale actions are refused before mock-driver writes. A post-recheck fault injection also tests a context change queued before the dispatch continuation. In a focused test-only ablation that omits only the tuple-equality check, six stale product/run changes reach mock actuation, while independent checks still refuse the other six cases.

The paper makes two contributions:

1) It specifies and implements a modular task-to-device integration contract that joins collaborative tasks and the registered-harness interface to turn-time production context, per-call operation checks, semantic I/O, config-

ured protocol drivers, and service-owned records. For approved recipe actions, the implementation binds approval to a line–product–recipe–run tuple and rechecks it after asynchronous authorization validation, immediately before entering software dispatch. This is a software-state guard, not an atomic lock across device I/O. It distinguishes prompt grounding, dispatch authority, device receipt, and later process observation; it is an implementation contribution, not a new protocol, safety-verification method, or process-control algorithm.

2) It evaluates the integration contract through registered-harness tasks, a deterministic harness replay, direct operation calls, and mock-state context-change tests across cast-film and annealing simulator mappings. The measurements distinguish route completion and record linkage from process target attainment; a focused ablation isolates the contribution of the approval-time context-tuple check.

Section II reviews related industrial-agent and integration work. Section III presents the design rationale, architecture, and cross-module execution flow; Section IV explains module responsibilities and their integrated operation. Section V describes the experiments and results, and Sections VI and VII discuss the evidence limits and conclude the paper.

II. Related Work

This section relates AgentWorkShop to plant-facing industrial agents and to production and automation abstractions for context, tool access, and operational constraints. Table I compares integration and execution features described in selected industrial-agent architectures, including systems with adjacent but distinct process-control objectives. The comparison is focused rather than exhaustive; a feature marked as not specified was not described in the cited study and is not inferred to be absent. Studies were included in this comparison when they describe an industrial agent's production context, plant-facing operations, or an action, approval, or evidence pathway. This criterion defines the table's scope; it is not a comprehensive survey or a performance ranking. Feature entries were coded from the cited study's full text only when the feature was explicitly described; otherwise the cell is marked "Not specified." Evaluation descriptions are transcribed at the level reported by each source, without normalizing sample units, success criteria, or metrics across studies.

A. Industrial Agents and Plant Integration

Industrial-agent systems, including large language model (LLM)-based systems, span end-to-end automation control, supervisory services, programmable logic controller (PLC) interfaces, manufacturing coordination, and operator-facing machine access. Xia et al. present an LLM agent framework for end-to-end industrial automation control, using structured prompts and event-driven context for production planning and operation [**?**]. Huang et al. present a multimodal architecture whose stated focus is automated control generation for industrial cyber-physical systems [**?**]. Biskupovic et al. study agent-based supervision for service-oriented industrial systems [**?**]; Agents4PLC and InstructMPC address PLC code generation and human–LLM context-aware control, respectively [**?**], [**?**]. Manufacturing studies consider dynamic resource-capability exploration [**?**], multi-agent shop-floor coordination [**?**], and digital-twin-supported fault handling [**?**]. Natural-language machine access [**?**] and real-time LLM control of a robot-operated serial line [**?**] extend this work to operator interfaces and robotic-line control. Together, these studies address different operational contexts, interfaces, and allocations of responsibility between agents, operators, and controllers.

The closest architecture-level comparison is Xia et al., who connect cognitive, digital-twin, and physical-automation layers and organize twin information as state snapshots, event messages, and plan sequences; their case studies address robotic automation and process simulation [**?**]. Other nearby designs expose factory-module operations through MCP with state-based tool filtering, plan constrained manufacturing steps, or compose semantic recipes from OPC UA skills [**?**], [**?**], [**?**]. These studies describe important parts of plant-facing integration. The recent systems in Table I also combine agent execution with approval, industrial data, or audit records. Graul et al. ground proposals in the Asset Administration Shell (AAS) and route approved actions through OPC UA Methods, with signed approvals and audit logging [**?**]. Schall separates advisory language-model agents from process and instrumentation diagram (P&ID)-grounded verification and execution, and evaluates structural checks across five simulated process scenarios [**?**]; its context includes supervisory control and data acquisition (SCADA) and standard operating procedure (SOP) information. Lin and Chen build a governed sensor–MES–ERP digital thread for closed-loop monitoring and control, evaluated on real manufacturing records [**?**]. These works address asset-specific approval, process validation, or closed-loop analytics. AgentWorkShop addresses the complementary software-integration problem of composing task and direct-operation routes with production context, semantic I/O, configured drivers, and service-owned records. Its implementation also rechecks the line–product–recipe–run tuple captured by a pending recipe approval immediately before software dispatch, while keeping command receipts distinct from later process observations. This is a scoped implementation contract, not a claim that other systems lack equivalent mechanisms. The evaluated unit is one named operation composed with distinct task and direct-call entry routes, two scene-specific protocol mappings, and separate service records. The evidence tests whether this composition executes and preserves its route-specific links; it does not measure reduced integration effort or superiority over another platform. Table I summarizes only features explicitly described in the cited studies.

The operation checks complement safety-aware decision support [**?**] but do not replace independent process protection.

B. Production Context, Agent Interfaces, and Automation

Industrial integration also depends on abstractions that are not specific to agent architectures. The Model Context Protocol (MCP) specifies interoperable tool discovery and call mechanisms [**?**]; the operation-level scope, policy, grant,

TABLE I
FEATURE-LEVEL COMPARISON OF SELECTED PLANT-FACING INTEGRATION STUDIES. ENTRIES SUMMARIZE REPORTED DESIGN SCOPE AND EVALUATION, NOT PERFORMANCE.

System	Context and operation checks	Device-facing route	Reported evaluation
Xia et al. [**?**]	Cognitive, twin, and physical layers; snapshots, events, plans	Digital-twin/physical automation	Robotic automation and process-simulation cases
Köhle et al. [**?**]	Factory agents with injected live state; state-based tool filtering	MCP exposes OPC UA methods; MQTT coordination	Nine simulated production challenges
Vieira da Silva et al. [**?**]	Constrained LLM planning; operation checks not specified	MCP-exposed manufacturing functions	Laboratory multi-step process
Köhle et al. [**?**]	AI-assisted recipes from OPC UA skills; checks not specified	OPC UA skills and virtual commissioning	Unity assembly-workstation proof of concept
Graul et al. [**?**]	AAS-grounded context; signed human approval	PLC-validated OPC UA Methods; immutable audit log	Two laboratory PoCs; single scripted runs
Schall [**?**]	P&ID, SCADA, time-series, SOP; topology and rule checks	Advisory proposal with gated execution	Five simulated scenarios; 43 proposals and 50 LLM runs
Lin and Chen [**?**]	Sensor–MES–ERP thread; SOP-, risk-, capability-aware gating	Edge monitoring and closed-loop actions	25,275 real records; operational metrics
This work	Collaborative tasks and direct calls; approval-bound context rechecked before recipe dispatch	Shared dispatcher; semantic nodes resolve to configured DAQ/DCW drivers	Registered-harness and direct routes across two simulator mappings; no attainment claim

and binding checks discussed here are AgentWorkShop's own dispatcher rules, distinct from MCP transport authorization. In parallel, ISA-88 and ISA-95 describe recipes and production operations [**?**], [**?**]. Run-to-run methods connect process observations to successive production runs [**?**]. At the agent level, AgentSpec and CaMeL address tool behavior and capability-based information flow [**?**], [**?**]. Together, these works provide complementary abstractions for production context, tool interaction, and information flow. AgentWorkShop composes such abstractions with its task runtime, operation checks, semantic I/O, and route-specific records; this paper evaluates selected paths through that composition rather than proposing a replacement for the underlying standards.

III. SYSTEM DESIGN

AgentWorkShop is designed for industrial tasks whose agents, production contexts, interfaces, and process signals vary between deployments. Its architecture organizes these variations around shared task and operation services, checks each proposed operation against the active production context before device routing, and retains route-specific evidence that distinguishes a command receipt from a measured process response.

A. Design Rationale and Requirements

Five requirements—scene variability, harness variability, operation authority, outcome observability, and extensibility—organize six design rules (D1–D6). These are deployment constraints drawn from the target workflows. D1 stores scene settings as versioned data; D2 provides registered harness, tool, and driver interfaces; D3 separates proposal from execution; D4 treats the tool catalog as discovery rather than permission; D5 routes actuator writes through the common DCW admission point; and D6 retains evidence by operation.

The requirements trace to implementation as follows. Scene variability maps to D1, scene blueprints, and context resolution; harness variability and extensibility map to D2, the registered harness and driver interfaces; operation authority maps to D3–D5, the shared dispatcher, grants, bindings, and write admission; outcome observability maps to D6, route-specific receipts, and observations. These interfaces are design extension points; the evaluation in Section V tests selected registered-harness and direct-operation routes, not general harness interchangeability. AgentWorkShop supplies task and operation services; agent strategy and independent process protection remain outside this design.

B. Layered Architecture

Fig. 1 summarizes the access routes, shared dispatch path, production services, semantic I/O, configured drivers, and service ownership of records. Task and chat routes enter the collaboration band, which contains workspaces, channels, task queues, shared memory, mailboxes, task templates, and harness adapters. Direct REST tool calls and the stdio MCP operation bridge can call the shared dispatcher without creating a task or invoking a harness; MCP management and task methods remain on their service routes. Direct calls have no agent-turn prompt context, so handlers resolve current route state from the request and services at invocation. The

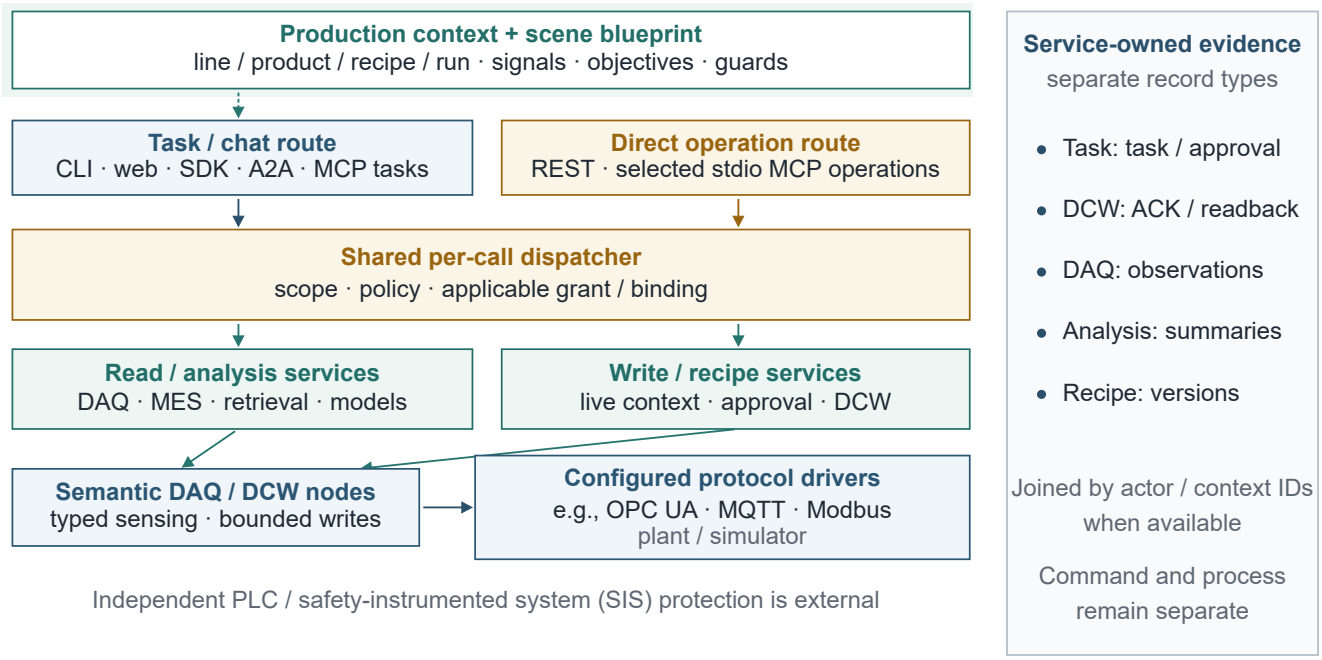


Fig. 1. AgentWorkShop architecture and record ownership. Task/chat calls pass through the listed access surfaces and a registered harness; direct REST and selected stdio-MCP operation calls enter the shared dispatcher, while MCP management/task methods use their service routes. R22 evaluates direct REST operation calls; the stdio-MCP route is shown but not evaluated. Recipe writes recheck the approved line–product–recipe–run tuple; each service retains its own records. Independent programmable logic controller and safety-instrumented system protection remains external.

governance band projects an operation catalog and checks each invocation. The context band assembles bound-line and active-run information for prompt grounding, while operation handlers resolve the live state relevant to their route. The service band provides acquisition, write control, analysis, retrieval, and optional model services. The field-I/O band resolves semantic nodes to built-in or plugin drivers for a plant or simulator.

Each band has a distinct responsibility. Task/chat routes enter the task runtime; direct operation routes enter the dispatcher; and both use shared operation checks where applicable. A harness adapter supplies agent-specific interaction on task paths, semantic nodes hide protocol details from operations, and the driver registry provides the device-facing extension point. Owning services retain task, command, observation, and version records separately and link them to available actor and production context. No single status conflates these outcomes.

C. Cross-Module Execution Flow

Fig. 2 summarizes a task turn from entry through operation execution, with route-specific evidence retained by owning services. Registered harness adapters receive a scoped tool catalog during setup and supported registry refreshes; the runtime assembles prompt-time production context for each agent turn. This context is grounding information, not a task-pinned transaction. For each returned tool call, the shared dispatcher rechecks invocation scope and channel policy and, for industrial operations, current grant and binding conditions before handler selection. The selected handler then applies route-specific guards, active-run checks, approval revalidation, and device-I/O steps as applicable. Command receipts and later process observations remain distinct records.

For an industrial operation u by agent a in channel c, define the shared dispatch condition as

$$\mathcal{A}(a,c,u) = \bigwedge_{X \in \mathcal{C}(u)} X(a,c,u), \tag{1}$$

where S, P, G, and B denote invocation scope, current channel policy, runtime grant, and the agent's binding to referenced resources, respectively. $\mathcal{C}(u) \subseteq \{S,P,G,B\}$ is the set of predicates declared for operation u; it always contains S and P. The grant predicate G is omitted only for an explicitly grant-exempt industrial tool, and B is included when the operation declares resource references. A declared no-reference operation omits B. Each included check must pass before handler selection. The policy predicate includes channel tool selection, profile, and duty-separation checks where configured. This condition is necessary for handler selection, not sufficient for actuation: the selected route applies its own guards and approval, then rechecks authority, context, and applicable limits before dispatch.

Fig. 3 gives reference pseudocode for the registered-harness task path, not one literal orchestrator function. Direct REST tool invocation bypasses the task runtime and harness but enters the same shared dispatcher. Route-specific timing, approval, and evidence rules are detailed in Section IV.

IV. Module Functions and Integration

The architecture in Fig. 1 is implemented as cooperating modules rather than a single monolithic agent. Shared interfaces connect task execution to configuration, industrial

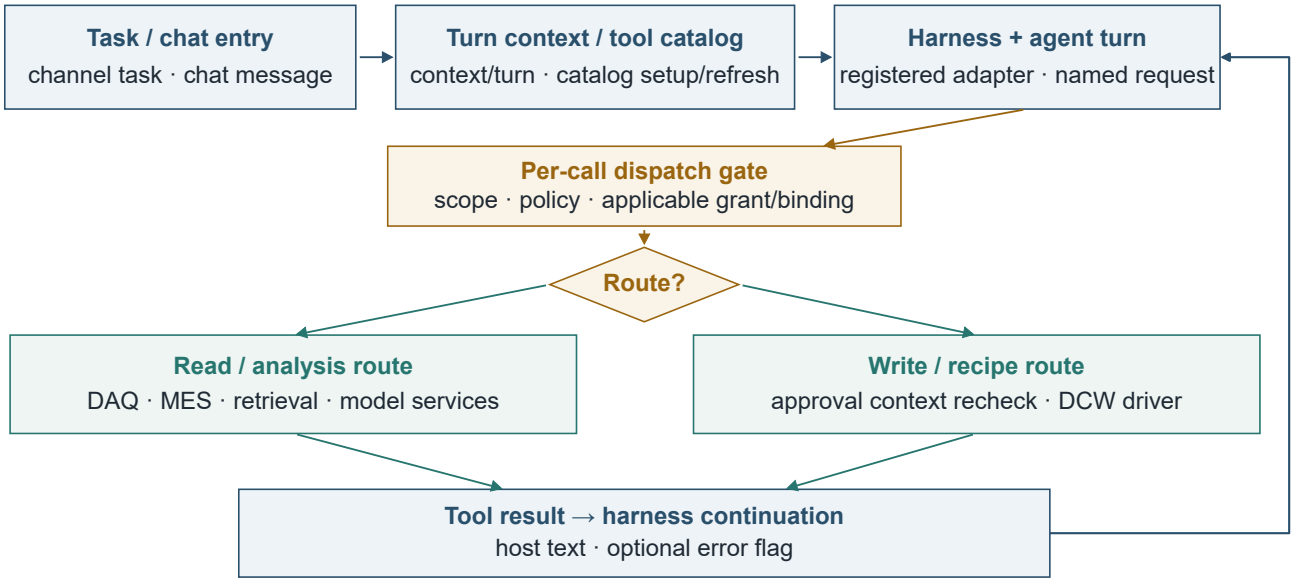


Fig. 2. Registered-harness task flow. Context is assembled per turn; the scoped tool catalog is loaded at setup or refresh. The dispatcher checks scope, policy, and applicable grants and bindings, then returns each tool result to the active harness turn. Direct REST and selected stdio-MCP operations enter the shared dispatcher without a task. In R20, the follow-up probe, ingest, and run-tagged query were initiated by the test runner outside the agent task. A command receipt remains distinct from a process observation.

Require: task q, channel c, bound agent a, registered scoped catalog T
Ensure: task result; any retained evidence remains with its owning service
$x_t \leftarrow \textsc{ResolvePromptContext}(c, q)$
$m \leftarrow \textsc{RetrieveMemory}(q)$
$p \leftarrow \textsc{AssemblePrompt}(q, x_t, m)$
$h \leftarrow \textsc{InvokeRegisteredHarness}(a, p, T)$
for all tool request u returned by h **do**
  $g \leftarrow \textsc{CheckScopeAndChannelPolicy}(a, c, u)$
  if not g **then**
    EmitToolErrorToHarness(h,u); **continue**
  end if
  if u is an industrial operation **then**
    $v \leftarrow \textsc{CheckGrantBindingOwnership}(a, u)$
    if not v **then**
      EmitToolErrorToHarness(h,u); **continue**
    end if
  end if
  $k \leftarrow \textsc{SelectHandler}(u)$
  $r \leftarrow \textsc{ExecuteRouteHandler}(k, a, c, u)$ ▷ handler resolves live state and may persist route-specific records
  EmitDeclaredToolResultToHarness(h,r)
end for
return task state and tool results

Fig. 3. Reference pseudocode for the registered-harness task path. Direct operation invocation is not shown. Evidence persistence is route-specific and remains with the owning service; the shared tool result does not carry a generic evidence envelope.

services, protocol drivers, and route-specific evidence. This section describes those responsibilities and the checks that preserve their boundaries.

The implementation scope is broader than the evaluated scope. Section V separates implemented interfaces from the routes and checks exercised here; an interface intended for extension does not by itself establish interchangeability or portability.

The extension contracts expose the data passed at each boundary. A harness call is normalized as `HostToolCall(toolName, arguments)` and returns `HostToolResult(text, isError)`. A DCW driver accepts an engineering value, tolerance, range, and driver configuration; its result separates write status, raw value, readback, and acknowledgment grade (readback-verified, transport-ack, or unverified). Each DAQ sample stores its node, timestamp, value, and state, with optional line, product, recipe, and run IDs. A write anchor records line/node, prior and requested values, source, actor, time, and optional run/task/approval IDs; recipe definitions retain separate versions. These fields keep tool errors, device receipts, observations, and production lineage inspectable as separate records.

DAQ and write-anchor records may omit line, product, recipe, run, task, or approval identifiers when a route does not supply them. For the R20 trace audit, the join keys are required: task ID, approval ID, recipe-trial run ID, active-run ID, and the run ID returned with the sample must agree with the corresponding route records. R21 checks the corresponding IDs on its deterministic fixture path. Thus the reported complete joins are requirements of those evaluated records, not a guarantee that every general record contains every identifier.

A. Task Runtime and Harness Adapters

Task and chat access surfaces create or join a workspace channel, where the task engine manages members, task queues, shared memory, and execution templates. Direct REST tool calls and selected stdio MCP operation calls can enter the shared dispatcher without creating a task or invoking a harness; MCP management and task methods use their corresponding services. A direct call has no agent-turn prompt context: its operation handler resolves live route state from request identifiers and service data at invocation. The harness registry instantiates an adapter behind a common agent interface; the prompt builder combines the task with the active scene brief, industrial context, channel profile, agent binding, and retrieved memory. Tool calls return to the shared dispatcher regardless of which registered harness produced them. The registry keeps engine-specific execution behind the common agent interface. Industrial context is assembled for each agent turn rather than pinned for the task lifetime, so prompt context is grounding information rather than write authorization. An approved recipe action separately captures the current line–product–recipe–run tuple when approval is requested and compares it again before dispatch. This approval-time check does not imply that the earlier prompt context remained unchanged.

B. Production Context and Configuration

Scene blueprints describe devices, signal maps, objectives, and process guards. A task resolves these definitions against its active line–product–recipe–run context. The channel profile selects the available capability family, the tool policy narrows the operation catalog, and the agent binding scopes the nodes, controls, and recipe operations available to that member. These filters can restrict one another but cannot broaden the underlying device range. The dispatcher re-reads policy at execution, so a cached catalog does not retain authority after a grant or policy change. The prompt builder assembles the agent's available bound-line and active-run context on each turn; this prompt-time context is grounding information and does not pin the complete tuple to the task.

An approved recipe action separately captures the active tuple when approval is requested. After the approval wait, the handler refuses software dispatch if the line, product, recipe, or run field has changed. This route-level check is separate from prompt-context assembly and does not lock physical I/O.

Specific production snapshot, trial, and MPC routes, together with Hybrid Twin training submissions, resolve and validate a versioned `SceneContract` at operation time. SceneContract enforcement is route-specific: the Twin physics-specification draft route can instead accept a caller-supplied scene description, and therefore does not validate or pin all prompt context.

C. Acquisition, Analysis, and Modeling Services

Semantic DAQ nodes expose typed values, engineering units, ranges, and sampling settings while hiding protocol-specific reads. A per-node DAQ runtime schedules acquisition and stores timestamped observations; the gateway aggregates node pipelines. MES-facing operations provide the configured production-data path. Analysis and retrieval services consume these records, while optional model, training, optimization, and digital-twin services are invoked through their registered operation handlers. These services complement the agent workflow; AgentWorkShop does not prescribe a single optimization strategy.

D. Semantic I/O and Actuation

The driver registry resolves a semantic node to a built-in or plugin driver. DAQ handlers use the node's acquisition interface; DCW handlers route a write or readback through the controller and the configured protocol driver. This separation lets recipes and agents refer to semantic controls without embedding Modbus, OPC UA, MQTT, HTTP, or MES transport details in their operation arguments. The supported driver mappings are implementation interfaces; only the mappings and workflows enumerated in Section V are experimentally exercised.

E. Shared Dispatch and Write Admission

Every named operation passes the dispatcher. Its ordered checks validate invocation scope and current channel policy (tool selection, profile, and applicable duty separation), then the agent's binding and runtime grant and ownership of referenced resources. A refusal returns the excluding condition. Denial persistence is selective: grant denials and selected resource-reference denials have explicit audit paths, whereas scope and channel-policy denials are not uniformly retained as durable audit events. A catalog entry therefore describes availability but does not authorize execution. Handlers for operations that require manual approval suspend for a decision, then revalidate authority and active limits before dispatch. For manual recipe operations, the handler checks the active run, candidate parameters, and route timing before approval; after the wait it rechecks current authority and active limits. For `recipe_apply`, a changed definition requires a new proposal. These route-specific checks complement, but do not replace, the per-call dispatcher checks.

All actuator writes converge on the DCW admission point. For an online node write, the admissible interval is

$$\mathcal{I}_n^{\text{online}}(t) = \mathcal{H}_n \cap \mathcal{B}_n \cap \mathcal{P}_n(t) \cap \mathcal{R}_n(t), \tag{2}$$

where $\mathcal{H}_n$ is the hard engineering range and $\mathcal{B}_n$, $\mathcal{P}_n$, and $\mathcal{R}_n$ are the baseline, active-product, and active-recipe limits. Missing optional limits impose no restriction, whereas an empty intersection rejects the request and identifies the excluding layer. Recipe dispatch applies the hard, baseline, and product limits to the replacement recipe rather than intersecting with the previous recipe window. The actuator path also enforces step, interval, and hold-window checks. These admission rules constrain commands; process objectives and coupled variable guards are assessed from later observations. The framework does not replace independent PLC or safety-instrumented protection.

F. Integrated Operation and Route-Specific Evidence

A recipe trial illustrates how the modules cooperate. The task runtime delivers a named `recipe_trial` call from the agent harness; the dispatcher checks the agent's recipe binding and execution scope; the recipe handler checks the active run, candidate parameters, and route timing; and a manual binding requests approval before the DCW controller sends the batch through its driver. The handler returns a per-parameter receipt. The caller then obtains DAQ observations and can request a separate update, apply, or rollback operation. A trial receipt confirms command handling; it does not by itself establish process attainment or persist a new recipe version.

The routes retain different records because they answer different questions. A driver acknowledgment reports command evidence; DAQ reports a process observation; an exploration summary records a bounded probe; and a recipe version records a saved definition. Their relation through actor and production context allows an execution to be inspected without treating these outcomes as interchangeable.

G. Route-Specific Lifecycles and Evidence

a) Timing and approval: Timing guards are route-specific. Online node writes enforce an effective minimum spacing of 60 s. Recipe dispatch uses a per-recipe interval anchored to approval, while trials also use a configurable per-line cadence (default 300 s). The recipe interval can refuse a proposal before an approval request is created. Checks occur at proposal or execution according to the operation, and experiment-specific pacing overrides are reported with the corresponding results.

Manual bindings create an approval request containing the actor, operation, candidate values, applicable limits, expiry, parameter bases, and the active line–product–recipe–run context. The default wait expires after 180 s and refuses execution. After approval, the executor rechecks authorization and compares the current context tuple with the approved tuple. For approved recipe writes and dispatched rollback, a final synchronous comparison follows the asynchronous grant check immediately before entering the DCW dispatch path. Changes visible at this software boundary are refused before a driver write. This check is not an atomic lock across network or device I/O and cannot cancel a driver call after it begins. A focused mock-state suite changes line, product, recipe, or run during approval for recipe apply, trial, and dispatching rollback; each mutation is refused before a driver write or new execution record. For `recipe_apply`, a changed recipe definition also requires a new proposal; an unchanged approval dispatches a copy of the approved parameter snapshot. A line-level recipe mode defaults to manual; changing it to automatic requires administrator confirmation. The approval record authorizes a workflow action and does not replace process protection.

Fig. 4(a) illustrates the four-layer interval in Eq. (2) with an archived RPM configuration; panel (b) shows a symbolic approval followed by context and authority rechecks. The plotted values are illustrative and are not R20 measurements.

b) Node probe: The bounded `optimization_explore` route is exposed to the AML optimization profile. It validates a bound control and DAQ scope, captures a baseline, routes a bounded write through DCW, and stores a later target window as a separate exploration summary. Its timing and approval behavior is route-specific; R20 does not exercise this route.

c) Recipe operations: `recipe_trial` checks the bound recipe and active run, then dispatches candidate overrides as a batch with per-parameter verification. The caller subsequently queries DAQ and may save a definition with `recipe_update`, restore one with `recipe_rollback`, or apply a version with `recipe_apply`. A trial does not save or apply a version, and endpoint verification is not a process-response measurement. Public direct-node control is closed; bounded probes and recipe operations are the configured write surfaces.

d) Command verification: Command confirmation and process response answer different questions. A driver ACK is graded as `readback-verified`, `transport-ack`, or `unverified`; a matching endpoint read supports the first grade, whereas transport or MES application confirmation supports the second. Recipe-batch verification is `verified`, `unverified`, or `failed`. If readback is absent or mismatched, up to three reads are attempted 2 s apart. Initial ACKs, final ACKs, and per-parameter verdicts are retained separately. A readback-verified setpoint does not establish target attainment.

e) Observation, verdict, and compensation: Recipe-path observation follows batch dispatch, and saving a definition, applying it, and observing process response remain separate operations. An open node record can request rollback after repeated breaches; missing samples do not count as breaches. Caller verdicts and compensation remain separate from the observation and subject to admission checks.

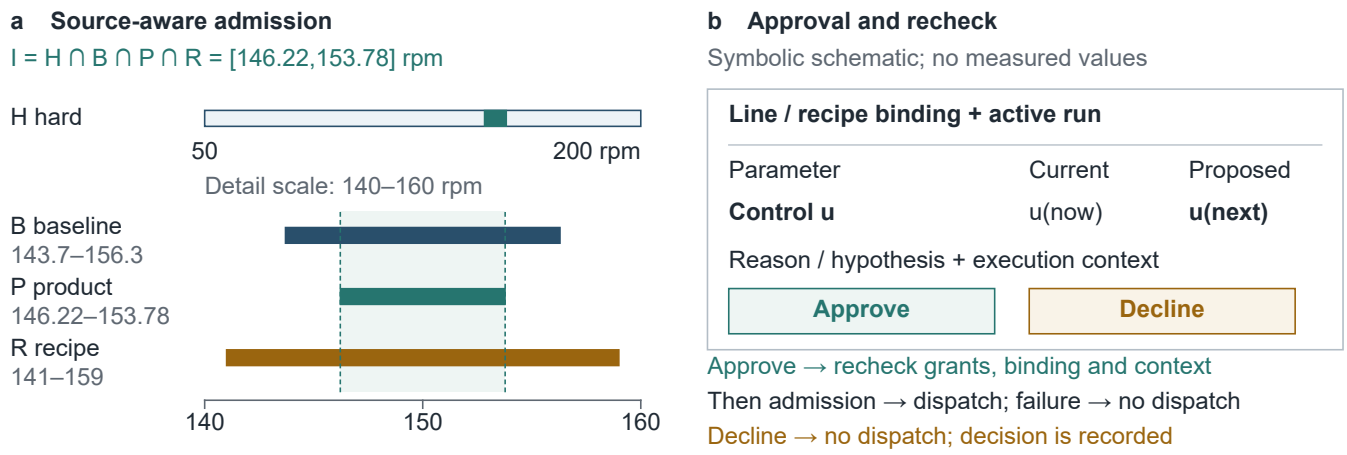


Fig. 4. Admission and approval boundaries. (a) Illustrative archived RPM configuration (not part of R20): online node interventions use the four-layer value interval of Eq. (2), whereas recipe dispatch omits the prior recipe window. (b) Symbolic human approval binds a proposed action and its context tuple; execution rechecks the active line–product–recipe–run context, authority, and applicable limits after approval. Approval does not reserve equipment state.

V. Experimental Evaluation

The evaluation asks whether registered-harness tasks and direct operation calls can execute bounded recipe actions across two simulator scenes with different signal and protocol mappings while retaining route-appropriate records. R20 is the primary result: six sequential tasks through one registered harness, three per scene on one simulator instance per scene, with task-to-approval-to-receipt- to-sample links. R21 is a deterministic fixture replay of the registered- harness path; R22 separately exercises the direct operation endpoint without a task or harness. All runs use repository simulators. Supplementary File S1 contains their separate records, scripts, and trace checks.

A. Evidence Units and Experimental Procedure

R20 uses the registered harness adapter named `omp` in cast-film and annealing scenes (`omp` is the project adapter key, not a network protocol), with three sequential task executions per scene on one simulator instance per scene. Each execution creates a new logical line, product, recipe, active run, channel, agent, and task on that shared scene simulator. Line start reapplies the recipe baseline (120 rpm or 730 °C), so the control read returns to baseline while the simulated process state carries forward. These are sequential software runs, not independent process replications. R21 and R22 use fresh isolated application/simulator environments and are reported separately from R20.

Table II summarizes the evaluated paths and their inference limits; Table III gives the six R20 process snapshots.

B. Registered-Harness Process Path (R20)

R20 ran the local v0.8.0 working-tree application with the registered `omp` adapter and GLM-5.3-Flash. Supplementary File S1 lists the source reconstruction requirements; credentials are omitted and external model decisions are not deterministic. Cast-film used OPC UA control and MQTT thickness sensing; annealing used Modbus TCP for control and sensing. In each scene the model selected one value from a prevalidated candidate set for a bounded `recipe_trial`, and a synthetic administrator approved it. Each trial produced one readback-verified receipt. Twenty seconds later, the runner probed and ingested one sample, then queried it by active-run ID through the isolated SQLite fallback because the isolated MQTT queue and Timescale endpoints were unreachable. All six tasks passed 76 runner assertions. The S1 per-execution trace matrix links task, approval, trial receipt, and active-run sample IDs; trial-run and sample run IDs differ but join through the task and approval. These experiment-level joins do not imply that every service record persists every link field. Task state was checked after sampling, so completion-to-sample latency is unavailable. Table III gives the values. Each pre-task value is a fresh control/process read during logical-run initialization, separate from the post-receipt probe. Each subsequent pre-task read followed the preceding post-trial probe by about 1.0–1.1 s; the simulator state persisted while each new line reapplied its baseline. These are successive snapshots, not replicate pairs. All six samples were outside target; one probe per task does not measure process response or normal online DAQ.

C. Deterministic Fixture Replay (R21)

R21 used a deterministic transport fixture for the registered `omp` adapter, without an LLM decision. One task per scene exercised approval, verified receipt/readback, and a run-tagged sample; all 31 runner checks, including cleanup, passed. This is a replay of the same harness route, not an independent agent or process replication, and is reported separately from R20.

D. Direct Operation Endpoint (R22)

R22 used a fresh isolated environment. A worker identity issued one direct `recipe_trial` call per scene through `/api/workshop/agent-tools/invoke`, without a task, harness transport, or external model. Synthetic approval produced a 1/1 verified receipt and matching readback for each call. After 20 s, the runner queried one active-run-tagged sample per scene; Table II reports both values and target bands. All 19 checks passed.

TABLE II
EVALUATION PATHS, RECORDED OUTCOMES, AND INFERENCE LIMITS. THE ROUTE COUNTS ARE KEPT SEPARATE BECAUSE THE RUNS USE DIFFERENT ENTRY PATHS AND EXECUTION MODES.

Path	Execution	Recorded outcome	Inference limit
R20	Six sequential tasks through one registered harness; three per scene.	6/6 task-to-sample joins and verified receipts; all probes outside target.	One simulator and one probe per scene/task; no attainment claim.
R21	One deterministic fixture task per scene on the same harness route.	Two fixture traces; 31 runner checks passed.	No external model decision or process replication.
R22	One direct `recipe_trial` call per scene via REST; no task, harness, or model.	2/2 verified receipt/readbacks; run-tagged samples were 46.37 μm (48–52 μm target) and 675.20 °C (700–760 °C target), both below band; 19 checks passed.	No task-to-sample join or cross-harness claim.
Context tests	Stale-context mutations on three recipe routes; handler test and test-only ablation.	All injected stale contexts refused before writes; ablation isolates product/run tuple checks.	Mock/temporary state; not a platform baseline or physical safety test.

TABLE III
BOUNDED RECIPE-TRIAL SETTINGS AND POST-READBACK SIMULATOR PROBES. ALL SIX SAMPLES REMAINED OUTSIDE THEIR TARGET BANDS.

Case	Pre-task value / target band	Control: start→trial	Sample
Film-1	55.82/48–52 μm	120 → 125 rpm	45.69 μm
Film-2	45.69/48–52 μm	120 → 130 rpm	46.69 μm
Film-3	46.69/48–52 μm	120 → 125 rpm	45.69 μm
Anneal-1	675.40/700–760 °C	730 → 745 °C	679.60 °C
Anneal-2	678.30/700–760 °C	730 → 745 °C	677.70 °C
Anneal-3	676.90/700–760 °C	730 → 745 °C	678.10 °C

E. Software Contract Checks

The local unit suite passed 566/566 tests across seven suites. In the 20-case approval suite, twelve injected line, product, recipe, or run changes across three recipe routes were refused before mock writes or new execution records. One queued post-recheck context change was also refused before dispatch. In a test-only ablation that removed tuple equality but retained authority and limit checks, six product/run changes caused twelve mock writes and six execution records; independent checks still blocked line/recipe changes. This component ablation isolates the tuple check's incremental software-state coverage; it is not a platform comparison or safety evaluation. Tests use mock or temporary state and do not establish atomicity during device I/O or test end-to-end DAQ tag failures.

VI. DISCUSSION

Across task and direct-call routes, the results show path feasibility and record traceability across two simulator scenes and approval-bound context changes. They do not establish process performance or comparative platform value. This software-integration evidence complements studies of asset-specific approval, process validation, and closed-loop analytics [**?**], [**?**], [**?**].

Each scene used one simulator instance and one probe per task, so independent replication, convergence, and process-quality gains remain untested. The evaluation did not test a second registered harness, online DAQ, configuration reuse, physical transfer, or a comparative baseline. The tuple check guards software dispatch but cannot lock physical I/O or external process state. Synthetic approval does not test operator performance [**?**], [**?**], [**?**]. Operation restrictions do not establish deployment security [**?**], [**?**], [**?**] or replace PLC/SIS safeguards [**?**]. Future work requires a defined platform comparison and independent simulator and physical-plant trials.

The positive-path records evaluated here do not constitute a complete durable denial ledger. In particular, scope and channel-policy refusals are not uniformly persisted, so auditability claims are limited to the route records and denial classes explicitly described in Section IV.

VII. CONCLUSION

AgentWorkShop connects collaborative tasks and direct operation calls to production context, checked operations, semantic I/O, and service-owned records. The evaluation verifies selected registered-harness and direct operation paths across two simulator mappings. Mock-state tests confirm refusal when approved context changes before dispatch; the ablation isolates added refusal coverage for product and active-run changes. These results validate selected software behavior, not platform-level comparative integration value or process performance.

ACKNOWLEDGMENT

OpenAI ChatGPT revised Sections I–VI and drafted Python for Figs. 1, 2, and 4. OpenAI image generation produced early concepts for Figs. 1 and 2; final vectors were redrawn and implementation-checked. AI generated no experimental records or measurements. The authors verified all submitted content.